关于第一次大作业的若干提示

赵银亮

2026.4.3

示例针对 fact 程序的基于 σ-DFA(<Q2ARM 左列>)的指令选择实现及测试过程，分为四步：构建 σ-DFA 并程序实现，能以任务程序作为输入，输出 ARM 程序；手工补全为完整汇编程序；汇编、链接、运行；用 GDB 查看运行过程。
已知任务程序：
fact@code=[LABEL l0; t1=2; IF n<t1 THEN l1 ELSE l2; LABEL l1; RETURN a; GOTO l3; LABEL l2; t2=n-1; t3=n*a; n=t2; a=t3; GOTO l0; LABEL l3;]
新版 QTAC 及 Q2ARM 见文档末尾。

第一步为构建 σ-DFA 并程序实现。

如以下提示，分为 4 遍完成。
第 1 遍输出：

```
(KEY,LABEL) (l,l0)(;,_) (temp,t1) (=,_) (k,2) (;,_) (KEY,IF) (d,n) (rop,<) (temp,t1) (KEY,THEN) (l,l1) (KEY,ELSE) (l,l2) (;,_) (KEY,LABEL) (l,l1) (;,_) (KEY,RETURN) (d,a) (;,_) (KEY,GOTO) (l,l3) (;,_) (KEY,LABEL) (l,l2) (;,_) (temp,t2) (=,_) (d,n)(aop,-) (k,1) (;,_) (temp,t3)(=,_) (d,n) (aop,*) (d,a) (;,_) (d,n)(=,_)(temp,t2) (;,_) (d,a)(=,_)(temp,t3) (;,_) (KEY,GOTO) (l,l0) (;,_) (KEY,LABEL) (l,l3) (;,_)
```

第 2 遍输出：

```
(T_LABEL,(l,l0))
(T_MOVE,a is k,(temp,t1),(k,2))
(T_IF_LABEL,l1=l3,(d,n),(rop,<),(temp,t1),(l,l1),(l,l2))
(T_RETURN,a is q,(d,a))
(T_GOTO,(l,l3))
(T_LABEL,(l,l2))
(T_AOP,a is k,(temp,t2),(d,n),(aop,-),(k,1))
(T_AOP,a is q,(temp,t3),(d,n),(aop,*),(d,a))
(T_MOVE,a is q,(d,n),(temp,t2))
(T_MOVE,a is q,(d,a),(temp,t3))
(T_GOTO,(l,l0))
(T_LABEL,(l,l3))
```

第 3 遍输出：

```
l0:
```

MOV X_{t1}, #2;

CMP X_n, X_{t1}; B.GE l2; l1:

MOV X0,X_a; RET;

B l3;

l2:

SUB X_{t2},X_n,#1;

MUL X_{t3},X_n,X_a;

MOV X_n, X_{t2};

MOV X_a, X_{t3};

B l0;

l3:

第 4 遍输出：其中 X_{t1}=X9，X_n=X10，X_a=X11，X_{t2}=X12，X_{t3}=X13，其它不变。

l0:

MOV X9, #2;

CMP X10, X9; B.GE l2; l1:

MOV X0, X11; RET;

B l3;

l2:

SUB X12,X10,#1;

MUL X13,X10,X11;

MOV X10,X12;

MOV X11,X13;

B l0;

l3:

第二步手工添加代码形成完整的 ARM 程序。

```
.global _start

.section .text

/* ========================================
    主程序入口
    ======================================== */
_start:
    /* 1. 准备参数 */
    MOV X10, #5          // 对应代码中的 X10 (n)
    MOV X11, #1          // 对应代码中的 X11 (a)

    /* 2. 调用函数 */
    BL my_func
```

```
    /* 3. 退出程序 (Linux 系统调用) */
    // 如果程序正常返回，X0 中会有结果
    MOV X8, #93             // sys_exit
    SVC #0

/* ========================================
   第一步生成的 ARM 代码 (原封不动)
   ======================================== */
my_func:
l0:
    MOV X9, #2;
    CMP X10, X9;
    B.GE l2;

l1:
    MOV X0, X11;
    RET;
    B l3;               // <--- 逻辑残留：死代码，RET 后无法执行

l2:
    SUB X12,X10,#1;
    MUL X13,X10,X11;
    MOV X10, X12;
    MOV X11, X13;
    B l0;

l3:
    B  l3;              // <--- 逻辑残留：作为程序的“陷阱”，以后有用处（老师注）
```

第三步：运行并查看结果

```
# 1. 汇编
aarch64-linux-gnu-as -o program.o program.s

# 2. 链接
aarch64-linux-gnu-ld -o program program.o

# 3. 运行并查看返回码 (应该是 120)
./program
echo $?
```

第四步：采用 GDB 了解执行过程

要使用 GDB 查看这段 ARM64 汇编程序的运行过程，我们需要通过设置断点、单步执行和查看寄存器来观察 X10（计数器）和 X11（累积结果）的变化。
由于你的程序逻辑中包含了一个“死循环” (l3)，调试时我们会特别关注程序是如何跳转到那里并卡住的。
以下是完整的操作步骤指南：
1. 重新编译（带调试信息）
首先，必须加上 -g 参数重新编译，这样 GDB 才能识别代码行号。

```
aarch64-linux-gnu-as -g -o program.o program.s
aarch64-linux-gnu-ld -g -o program program.o
```

2. 启动 GDB
在终端中输入以下命令启动调试：

```
gdb program
```

3. 调试核心流程（GDB 命令速查）

进入 GDB 界面后（提示符为 (gdb)），你可以按照以下步骤操作：

第 A 步：找到入口并设置断点

我们首先在 _start（主程序入口）和 my_func（你的函数入口）处设置断点，防止程序一运行就结束。
. 调试核心流程（GDB 命令速查）
进入 GDB 界面后（提示符为 (gdb)），你可以按照以下步骤操作：
第一步：找到入口并设置断点
我们首先在 _start（主程序入口）和 my_func（你的函数入口）处设置断点，防止程序一运行就结束。

第 B 步：运行程序

输入 run (或简写 r) 启动程序。程序会在遇到第一个断点时暂停。

```
(gdb) run
```

第 C 步：单步跟踪（观察寄存器变化）

这是最关键的一步。当程序停在 my_func 时，使用 nexti (next instruction) 来一行一行地执行汇编指令，并使用 info registers 查看寄存器状态。
操作演示：
查看初始状态：
当程序停在 my_func 时，输入：

```
(gdb) info registers X10 X11
```

你应该看到 X10 是 5，X11 是 1。

单步执行循环：
不断输入 nexti (或 ni) 并观察寄存器。
当执行到 l2 标号附近的代码时（SUB, MUL），再次查看寄存器：
(gdb) info registers X10 X11 X12 X13

观察点：你会发现 X10 每次减 1，X11 变成了累乘的结果。
观察跳转逻辑：
当 X10 变为 1 时，执行 CMP X10, X9 和 B.GE l2。
此时条件不满足（1 不大于等于 2），程序会跳转到 l1。
观察返回与"残留逻辑"：
程序执行到 l1 的 RET 指令。
执行 RET 后，程序会返回到主程序的 _start 处继续执行退出系统调用。
注意：你代码中的 l3 标号（死循环）实际上不会被执行到，因为 RET 指令已经让函数返回了。B l3 是死代码。

常用 GDB 命令速查表

命令（及简写）	功能说明	针对本程序的用途
break (b)	设置断点	b my_func：在函数开始处停下，准备观察。
run (r)	运行程序	开始执行程序，直到遇到断点。
nexti (ni)	单步执行（汇编级）	最常用。一步一步看 SUB 和 MUL 怎么改变数值。
info reg	查看寄存器	info reg X10 X11：专门看你的变量变化。
display	自动显示	display/10i $pc：每次暂停时，自动显示接下来的 10 行汇编代码。
continue (c)	继续运行	直接跳到下一个断点（如果你想跳过繁琐的循环）。
quit (q)	退出 GDB	调试结束。

实战示例（模拟对话）

```
(gdb) b my_func                # 在函数入口打断点
Breakpoint 1 at 0x400078
(gdb) r                        # 运行
Starting program: ./program

Breakpoint 1, my_func () at program.s:15
15          l0:
(gdb) info reg X10 X11      # 查看初始值
X10                0x5          # n = 5
```

```
X11             0x1         # a = 1
(gdb) ni                    # 执行 MOV X9, #2
(gdb) ni                    # 执行 CMP X10, X9
(gdb) ni                    # 执行 B.GE l2 (跳转发生)
(gdb) ni                    # 现在在 l2 标号处
(gdb) ni                    # 执行 SUB X12, X10, #1
(gdb) info reg X12          # 查看 X12
X12             0x4         # 5 - 1 = 4
(gdb) ni                    # 执行 MUL X13, X10, X11
(gdb) info reg X13          # 查看 X13
X13             0x5         # 5 * 1 = 5
...                         # 继续 ni 直到循环结束
```

通过这种方式，你可以亲眼看到数据是如何在寄存器之间流动和计算的。

指令模板 Q2ARM

赵银亮
西安交通大学
2026.4.1

$\mathcal{L}$（QTAC 指令模板）	ARM64 指令生成模式
<T_RETURN>⟶RETURN a;	(a is q) MOV X0,X_a; RET; (a is k) MOV X0,#k; RET;
<T_PAR_CALL>⟶(PAR a;){0,7} q=CALL d,k;	MOV X0,X_{a1}; …; MOV Xk,X_{ak}; BL d; MOV X_q,X0;
<T_GOTO>⟶GOTO b;	(b is l) B l; (b is q) BR X_q;
<T_LABEL>⟶LABEL l;	l:
<T_IF_LABEL>⟶IF q ⟨rop⟩ a THEN l_1 ELSE l_2; LABEL l_3;	(l_3=l_2) CMP X_q, X_a; ⟨Jrop⟩ l_1; l_2: (l_3=l_1) CMP X_q, X_q; ⟨Jrop'⟩ l_2; l_1:
<T_IF>⟶IF q ⟨rop⟩ a THEN l_1 ELSE l_2;	CMP X_q, X_a; ⟨Jrop⟩ l_1; B l_2;
<T_LDR0>⟶q = M[b];	(b is q) LDR X_q, [X_b]; (b is l) LDR X_q, b;
<T_LDR2>⟶q_1 = q_2 + a; q_3 = M[q_1];	(q_1=q_2 ∧ a is k) LDR X_{q3}, [X_{q1},#k]!; (q_1≠q_2 ∧ a is q ∧ q^{last}) LDR X_{q3}, [X_{q1},X_a]; (q_1≠q_2 ∧ a is k ∧ q^{last}) LDR X_{q3}, [X_{q1},#a];
<T_STR0>⟶M[b] = q;	(b is q) STR X_q, [X_b]; (b is l) ADR ⟨Xcrs⟩,b; STR X_q,[Xcrs⟩];
<T_STR2>⟶q_1 = q_2 + a; M[q_1] = q_3;	(q_1=q_2 ∧ a is k) STR X_{q3}, [X_{q1},#k]!;

	($q_1 \neq q_2 \wedge a$ is $q \wedge q^{last}$) STR X_{q3}, [X_{q1},X_a]; ($q_1 \neq q_2 \wedge a$ is $k \wedge q^{last}$) STR X_{q3}, [X_{q1},#a];
<T_AOP>⟶q_1 = q_2 ⟨aop⟩ a;	(a is q) ⟨Xaop⟩ X_{q1}, X_{q2}, X_a; (a is k) ⟨Kaop⟩ X_{q1}, X_{q2}, #a;
<T_MOVE>⟶q = a;	(a is q) MOV X_q, X_a; (a is k) MOV X_q, #a;

⟨Jrop⟩：B.EQ、B.NE、B.GE、B.GT、B.LE、B.LT

⟨Jrop'⟩：B.NE、B.EQ、B.LT、B.LE、B.GT、B.GE

⟨Xaop⟩：ADD、SUB、MUL、SDIV　　⟨Kaop⟩：ADD、SUB

[*X*crs⟩虚拟寄存器，一次赋值仅一次使用

t 是临时变量，即⟨temp⟩；t^{last} 与 *t* 没有区别；

如果 *x* 是虚拟寄存器，那么 X_x 就是 *x*，否则 X_x 是虚拟寄存器。正体 X 开头的指物理寄存器。

中间语言 QTAC

赵银亮
西安交通大学
2026.4.1

d@code=[⟨QTAC-ins-list⟩]

⟨QTAC-ins-list⟩⟶⟨QTAC-ins⟩;|⟨QTAC-ins-list⟩⟨QTAC-ins⟩;

⟨QTAC-ins⟩⟶LABEL l | GOTO l | IF q ⟨rop⟩ a THEN l ELSE l

q⟶ d | ⟨temp⟩

a⟶q | k

⟨QTAC-ins⟩⟶PAR a | q = CALL d, k | RETURN a

⟨QTAC-ins⟩⟶ $q = q$ ⟨aop⟩ a | $q = a$

⟨QTAC-ins⟩⟶$q = M[b]$ | $M[b] = q$

b⟶q | l

⟨TAC-ins⟩⟶$q = \&b$ | NOP | q = CONVERT q, t

k 是立即数，k∈[0, 4095]；t 为 INT 或 FLOAT；l 为中间语言标号名，形式为 li；⟨temp⟩为临时变量名，形式为 ti。⟨aop⟩与⟨rop⟩分别是双目算术和关系运算符，优先级前者高于后者。

d⟶[a-z]+

⟨temp⟩⟶T_[0-9]+

l⟶L_[0-9]+

k⟶[0-9]+

⟨aop⟩⟶+|-|*|/

⟨rop⟩⟶<|<=|>|>=|==|!=

KEY⟶ LABEL | GOTO | IF | THEN | ELSE | PAR | CALL | RETURN | RETURN | ASG | AOP | LOAD | STORE | ADDR | NOP | CONVERT

⟨QTAC 程序⟩⟶(⟨QTAC-ins⟩;)+